

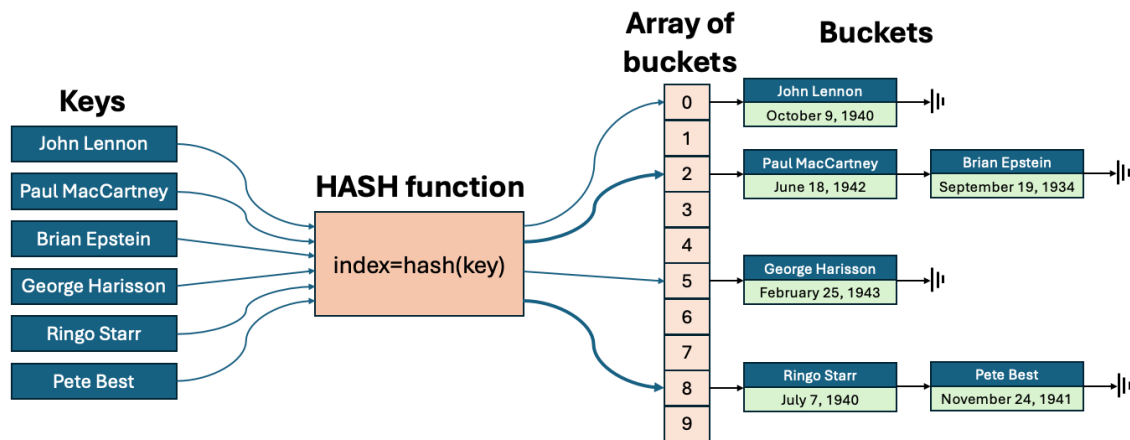
תרגיל 3, מבוא לתכנות מערכות, חורף תשפ"ו

הגשה בזוגות או ביחידים דרך המודל
התרגיל הוא להגשה עד ליום ראשון, 18/1/2026 בשעה 23:59

Hash Tables

מבנה הנתונים hash table שומר זוגות של מפתחות וערכים בדומה למילון (dict בפייתון), כלומר כל מפתח יכול להופיע בדיוק פעם אחת. המבנה הפנימי של hash table הוא של מערך של buckets כאשר כל bucket בנוי מרשימה מקושרת או מבנה נתונים אחר. על מנת לשמור זוג חדש של מפתח-ערך תקרא ה-hash table לפונקציית $index \rightarrow hash(key)$ שיוצרת מכל מפתח ערך index שקובע לאיזה bucket יכנס המפתח. בהכנסת זוג חדש של מפתח וערך תתבצע בדיקה בתוך ה-bucket האם המפתח כבר קיים. אם כן, יוחלף הערך הישן בערך החדש, אחרת יוכנס הזוג לרשימה המקושרת של ה-bucket.

הנה דוגמה סכמטית ל-hash table ששומר תאריכי ימי הולדת של אנשים שקשורים ללהקת החיפושיות. על פי הדוגמה ניתן להבין ש- $hash("Paul McCartney")=2$, $hash("John Lennon")=0$, וכו'.



בתרגיל זה תממשו מודול hash table גנרי בעזרת מודול LinkedList. בנוסף, תבצעו unit testing עבור כל אחד מהמודולים ותנסו מספר פונקציות hash ו-bucket array sizes.

משימות

הורידו מהמודל את הארכיב code.tar.gz שמכיל את קבצי הקוד של התרגיל. פתחו את הארכיב בתיקיית העבודה שלכם. כל הקבצים צריכים להיות חלק מהפרויקט עליו תעבדו.

1. שינוי המודול LinkedList

בחלק זה אין להשתמש ב-AI אלא אם כן מצויין אחרת.

1.1. ה-LinkedList שבו נשתמש ישמור את העותקים של ה-data שיתקבלו מהמשתמשת ולא ייצר עותק חדש. שנו את הקוד של ה-LinkedList בהתאם.

1.2. נרצה להוסיף את האפשרות לעבור על כל האיברים ב-LinkedList. הוסיפו שתי פונקציות לממשק ולמימוש:

- `LLFirst()` – מקבלת LinkedList, מחזירה את האלמנט שנמצא ב-Node הראשון ברשימה או NULL אם הרשימה ריקה. לא משנה את הרשימה המקושרת.
- `LLNext()` – מחזירה את האלמנט הבא ברשימה, או NULL כשסיימנו לעבור על כל האלמנטים ברשימה. לא משנה את הרשימה המקושרת.

על מנת לעבור על כל האלמנטים ברשימה מקושרת בשם ll נוכל להשתמש בקוד הבא:

```
for(Element e=LLFirst(ll); !e; e=LLNext(ll)) {  
    // Do something with e  
}
```

המשתמשת לא אמורה לשנות את הרשימה בזמן שמתבצע מעבר עם LLNext(). אם הרשימה משתנה התנהגות LLNext() היא לא מוגדרת ועשויה לגרום לשגיאת זמן ריצה.

1.3. הוסיפו ל-LinkedList תיעוד באנגלית שנוגע לדברים שחשוב שהמשתמשת תדע על הרשימה המקושרת שמשתמשת. למשל: שהמודול לא מייצר עותק אלא משתמש בעותק שהמשתמשת העבירה, ש-LLDestroy לא מוחקת את האלמנטים אלא רק את הרשימה, וכו'.

1.4. ממשו unit testing בקובץ בשם test_LinkedList.c. השתמשו ב-unity בשביל לבצע את הבדיקות. בחלק זה אני ממליץ על שימוש ב-AI: העבירו את קובץ ה-LinkedList.h לציאת החביב עליכם ובקשו שיצור unit tests בעזרת unity. עברו על הקוד, ודאו שאתם מבינים אותו, הוסיפו בדיקות שאתם חושבים שחסרות (אם יש כאלה), והריצו על LinkedList. תקנו את השגיאות כך שכל הבדיקות תעבורנה ללא תקלות.

1.5. בדקו בעזרת valgrind שהקוד רץ ללא דליפות זכרון

2. כתיבת המודול HashTable

בחלק זה מותר להשתמש ב-AI לצורך כתיבת הקוד, למשל ב-co-pilot בסביבת VSCode. האחריות על כל קוד שיכלול בתוכנית שלכם היא עליכם! אתם מצופים לדעת להסביר כל שורת קוד. מודול ה-HashTable שתיצרו צריך להיות גנרי עבור המפתח ועבור הערך. כלומר: מבנה הנתונים צריך לעבור עם כל סוג של מפתחות וערכים. מבנה הנתונים ייצור עותקים חדשים של המפתחות והערכים אותם הוא מקבל. 2.1. הוסיפו קובץ חדש לפרויקט בשם HashTable.h. הוסיפו לו את ההצהרות הבאות. תעדו את הפונקציות כנדרש בלבד.

- הגדרת הטיפוס Key כ-void*
 - הגדרת הטיפוס Value כ-void*
 - הצהרה על HashTable בהתאם ל-ADT
 - HTCreate: פונקציית Constructor ל-HashTable. מקבלת את מספר ה-buckets, מצביע לפונקציית ה-hash וכן כל פרמטר אחר שנחוצ, מחזירה HashTable
 - HTDestroy: מקבלת HashTable, משחררת את הזכרון שלו ושל כל המפתחות+ערכים שלו. לא מחזירה דבר
 - HTSize: מקבלת HashTable, מחזירה את מספר הערכים (זוגות מפתח+ערך) שבו
 - HTInsert: מקבלת HashTable, מפתח וערך, מוסיפה את הזוג אם המפתח לא נמצא או מחליפה את הערך הקיים של הזוג בערך שהועבר (ומשחררת את הערך הקודם) אם המפתח כבר נמצא. לא מחזירה דבר
 - HTRemove: מקבלת HashTable ומפתח, מוציאה את הזוג מה-HashTable, מחזירה את הערך ומשחררת כל זכרון שאין צורך בו יותר. מחזירה NULL אם המפתח לא קיים
 - HTSearch: כמו HTRemove אבל לא מוציאה את הזוג מה-HashTable
- 2.2. צרו את HashTable.c. ממשו את ה-struct שמכיל את המידע ל-HashTable וכן את הפונקציות שהגדרתם בממשק. חובה לממש את ה-buckets כרשימות מקושרות על ידי שימוש ב-LinkedList!
- 2.3. ממשו unit testing בקובץ בשם test_HashTable.c. השתמשו ב-unity בשביל לבצע את הבדיקות. העבירו את קובץ ה-HashTable.h לציאת החביב עליכם ובקשו שיצור unit tests בעזרת unity. עברו על הקוד, ודאו שאתם מבינים אותו, הוסיפו בדיקות שאתם חושבים שחסרות (אם יש כאלה), והריצו. תקנו את השגיאות כך שכל הבדיקות תעבורנה ללא תקלות.
- 2.4. בדקו בעזרת valgrind שהקוד רץ ללא דליפות זכרון

3. כתיבת main והשוואת פונקציות ה-hash

- בחלק זה מותר להשתמש ב-AI לצורך כתיבת הקוד, למשל ב-co-pilot בסביבת VSCode. השתמשו ב-main.c שנמצאת במודל. פונקציית main() צריכה לבצע את הדברים הבאים:
- 3.1. לבדוק נכונות של הפרמטרים
 - 3.2. לקרוא ל-srand() עם ה-seed שנבחר
 - 3.3. ליצור HashTable עם פונקציית ה-hash שנבחרה (hash1, hash2, hash3, כולן מוגדרות ב-misc.h) וה-bucket array size. גם המפתחות וגם הערכים צריכים להיות מטיפוס unsigned int
 - 3.4. לבצע n פעמים:
 - להגריל Key

- להגדיל Value
 - להוסיף את הזוג ל-HashTable
- 3.5 להדפיס את מספר האלמנטים שהוכנסו ע"י קריאה ל-HTSize()
- 3.6 להרוס את אובייקט ה-HashTable

4. כתיבת Makefile

כתבו makefile עבור ה-main.c שכתבתם ושאר המודולים. שם קובץ ההרצה צריך להיות prog. יש להריץ את gcc עם הפרמטרים -std=c99 -g -Wall -Werror. misc.o קומפל על שרת החוג ולכן תוכלו לשלב אותו בתוכנית רק אם תקמפלו אותה על שרת החוג.

5. הרצה

בדקו שהתוכנית רצה כמו שצריך בעזרת valgrind ובכל דרך אחרת שתעזור לכם למצוא באגים. הריצו את התוכנית על שרת החוג ובדקו את זמני הריצה שלה עבור פרמטרים שונים. הוסיפו בסוף main.c בהערה תשובות לשאלות הבאות:

1. איזה מפונקציות ה-hash עובדת הכי טוב?
2. האם גודל מערך ה-buckets משנה את הביצועים? באיזה מצבים?
3. בונים: עבור הפונקציה שעובדת הכי טוב, מדוע היא עובדת טוב? כמובן שאין צורך להבין את האלגוריתם שלה שאינו חשוב בפניכם.

לצורך מדידת זמני הריצה ניתן להשתמש בפקודה time של לינוקס:

```
$ time ./prog
Usage: ./prog <hash_function_number> <bucket-array-size> <seed> <n>
hash_function_number: 1, 2, or 3
bucket-array-size: positive integer
seed: positive integer
n: number of random items to insert

real    0m0.008s
user    0m0.001s
sys     0m0.003s
```

הפקודה נותנת 3 זמנים: real – זמן הריצה הכולל; user – זמן הריצה של התוכנית בלבד; sys – זמן הריצה של המערכת.

דגשים בכתיבת הקוד

- הקפידו על הנדסת תוכנה. דאגו שהקוד יהיה מסודר, צרו פונקציות עבור קטעי קוד שחוזרים על עצמם, הגדירו מאקרואים כשצריך, תעדו במקומות שבהם זה נדרש.
- לתוכנית אסור לקרוס עקב שגיאת זמן ריצה. חשבו על כל מקרי הקצה האפשריים, העזרו ב-AI לצורך כך. העזרו ב-AI בשביל לכתוב טסטים לתוכנית שלכם שיעזרו לוודא את נכונות החלקים השונים בתוכנית.
- לתוכנית אסור לגרום לדליפות זכרון. העזרו ב-valgrind שמותקן על שרת החוג בשביל לזהות דליפות זכרון.

טיפול בשגיאות

התוכנית תצא עם ערך שגיאה MEMORY_ALLOCATION_ERROR (מוגדר ב-misc.h) עבור שגיאות הקצאת זכרון. עקבו אחרי פורמט השגיאות כפי שמופיע ב-LinkedList.c.

הגשה

- ארוזו את כל קבצי ה-c, ה-h ו-Makefile (אבל לא את קובץ ההרצה או את קבצי ה-object) בארכיב בשם 73_W3_12345.zip והגישו דרך המודל. אם אתם מגישים בזוג אז רק אחד הזוגות צריך להגיש.

בהצלחה!